CS-462 – Programming Language Foundations
Lecture #16: Logical Relations

EPFL

1 Introduction

In Lecture 12, we saw that the simply-typed λ-calculus rules out the nonterminating expression $\Omega = (\lambda x.\, x\ x)\ (\lambda x.\, x\ x)$. To type the self-application $x\ x$, we would need a type τ with $\tau = \tau \to \tau'$, and there is no such type. In fact, *every* well-typed program terminates:

Theorem (Termination)**.** *If* $\vdash e : \tau$*, then there is a value* v *such that* $e \to^* v$.

In this lecture, we prove this theorem. The proof uses a technique called *logical relations*, which was invented by Tait in 1967.

Termination is a strong guarantee. Total languages such as Rocq (Coq), Agda, and Lean rely on it for their logical consistency. Unlike type soundness, it is not a syntactic property: progress and preservation tell us that a well-typed program never gets stuck, but they say nothing about how long it runs.

But the technique matters even more than the theorem. Logical relations are used to prove

- type soundness and parametricity (next lecture),
- program equivalence and compiler correctness,
- security properties such as noninterference, and
- the safety of libraries that use `unsafe` code in Rust.

2 The Language

To keep things simple, we work with the simply-typed λ-calculus over **unit**:

$$
\begin{aligned}
e &::= x \mid \lambda x : \tau.\, e \mid e_1\ e_2 \mid () \\
v &::= \lambda x : \tau.\, e \mid () \\
\tau &::= \mathbf{unit} \mid \tau_1 \to \tau_2
\end{aligned}
$$

with the standard call-by-value semantics:

$$E ::= [\cdot] \mid E\ e \mid v\ E \qquad \textsc{Context}\ \frac{e \to e'}{E[e] \to E[e']} \qquad \beta\textsc{-reduction}\ \frac{}{(\lambda x : \tau.\, e)\ v \to e\{v/x\}}$$

and the typing rules T-Var, T-Unit, T-Abs, and T-App from Lecture 12.

Definition (Halting)**.** We write $e\Downarrow$, pronounced "e halts", if $e \to^* v$ for some value v.

We will use the following standard facts from Lecture 12:

- *Determinism:* if $e \longrightarrow e_1$ and $e \longrightarrow e_2$, then $e_1 = e_2$.
  So if $e \longrightarrow e'$, then $e{\Downarrow}$ if and only if $e'{\Downarrow}$.
- *Preservation:* if $\vdash e : \tau$ and $e \longrightarrow e'$, then $\vdash e' : \tau$.
- *Canonical forms:*
  - if $\vdash v : \mathbf{unit}$, then $v = ()$;
  - if $\vdash v : \tau_1 \to \tau_2$, then $v = \lambda x : \tau_1 . e$ for some x and e.
- *Substitution:* if $\Gamma, x : \tau' \vdash e : \tau$ and $\vdash v : \tau'$, then $\Gamma \vdash e\{v/x\} : \tau$.

3 A First Attempt

Let's try to prove that if $\vdash e : \tau$ then $e{\Downarrow}$, by induction on the typing derivation.

Case T-Var: Impossible, since the typing context is empty.

Case T-Unit: Immediate, since () is a value.

Case T-Abs: Immediate, since $\lambda x : \tau . e'$ is a value.

Case T-App: Here $e = e_1\ e_2$, with

$$\vdash e_1 : \tau' \to \tau \qquad \text{and} \qquad \vdash e_2 : \tau'.$$

By the induction hypotheses, $e_1 \longrightarrow^* v_1$ and $e_2 \longrightarrow^* v_2$. By preservation and canonical forms, $v_1 = \lambda x : \tau' . e'$. So

$$e_1\ e_2 \ \longrightarrow^*\ (\lambda x : \tau' . e')\ e_2 \ \longrightarrow^*\ (\lambda x : \tau' . e')\ v_2 \ \longrightarrow\ e'\{v_2/x\}$$

Now we would like to show that $e'\{v_2/x\}{\Downarrow}$. But we cannot use an induction hypothesis:

- $e'\{v_2/x\}$ is not a subexpression of e;
- its typing derivation is not part of the derivation for e; and
- it may even be *larger* than e, so induction on the size of e does not help either.

The problem is that the induction hypothesis for e_1 tells us that it halts, but it tells us nothing about what happens when we *apply* it to an argument. We need a stronger induction hypothesis.

4 Logical Relations

For an expression of function type, we need to know not just that it halts, but also that applying it to an argument that halts gives a result that halts. But for higher-order functions, the arguments and results are themselves functions. So we need the same stronger property for them, and so on.

The solution is to define the property by induction on the *type*. A family of predicates indexed by types is called a *logical relation*. Here, it captures *hereditary termination*.

Definition (Logical relation)**.** We define the predicate $R_\tau(e)$ by induction on τ:

$$\begin{aligned} R_{\mathbf{unit}}(e) \quad &\text{iff} \quad \vdash e : \mathbf{unit} \text{ and } e{\Downarrow} \\ R_{\tau_1 \to \tau_2}(e) \quad &\text{iff} \quad \vdash e : \tau_1 \to \tau_2 \text{ and } e{\Downarrow} \text{ and} \\ &\qquad \text{for every } e', \text{ if } R_{\tau_1}(e') \text{ then } R_{\tau_2}(e\ e') \end{aligned}$$

This definition is well-founded, since the clause for $R_{\tau_1 \to \tau_2}$ only uses R_{τ_1} and R_{τ_2}, which are at smaller types. The clause for functions resembles a logical implication: related arguments are mapped to related results. This is why these relations are called "logical".

The proof of termination now follows from two lemmas. The first says that related expressions have the property we care about:

Lemma (Adequacy)**.** *If $R_\tau(e)$, then $e{\Downarrow}$.*

Proof. By cases on τ.

Case $\tau = \mathbf{unit}$**:** By definition, $R_{\mathbf{unit}}(e)$ means that $\vdash e : \mathbf{unit}$ and $e{\Downarrow}$. In particular, $e{\Downarrow}$.

Case $\tau = \tau_1 \to \tau_2$**:** By definition, $R_{\tau_1 \to \tau_2}(e)$ means that $\vdash e : \tau_1 \to \tau_2$, $e{\Downarrow}$, and e maps related arguments to related results. In particular, $e{\Downarrow}$. □

The proof is immediate because we built halting into every clause of the definition. Note that it is by cases on τ, not by induction: no induction hypothesis is needed.

The second lemma says that well-typed expressions are related:

Lemma (Fundamental lemma, first version)**.** *If $\vdash e : \tau$, then $R_\tau(e)$.*

Before we try to prove it, we need a lemma about evaluation.

Lemma (Closure under evaluation)**.** *If $\vdash e : \tau$ and $e \to e'$, then*

$$R_\tau(e) \quad \textit{if and only if} \quad R_\tau(e').$$

Proof. By induction on τ.

Case $\tau = \mathbf{unit}$**:** By definition, $R_{\mathbf{unit}}(e)$ means $\vdash e : \mathbf{unit}$ and $e{\Downarrow}$, and similarly for e'.

($\Rightarrow$) Suppose $R_{\mathbf{unit}}(e)$. Then $\vdash e' : \mathbf{unit}$ by preservation. And since $e{\Downarrow}$ and $e \to e'$, determinism gives $e'{\Downarrow}$. So $R_{\mathbf{unit}}(e')$.

($\Leftarrow$) Suppose $R_{\mathbf{unit}}(e')$. Then $\vdash e : \mathbf{unit}$ by assumption. And since $e \to e'$ and $e'{\Downarrow}$, we have $e{\Downarrow}$. So $R_{\mathbf{unit}}(e)$.

Case $\tau = \tau_1 \to \tau_2$**:** First, a fact we will use in both directions. Let e'' be any expression with $R_{\tau_1}(e'')$. Then $\vdash e'' : \tau_1$, so

$$\vdash e\ e'' : \tau_2 \qquad \text{and} \qquad e\ e'' \to e'\ e'', \tag{$*$}$$

where the step uses the evaluation context $E\ e''$. So the induction hypothesis at τ_2 applies to $e\ e''$ and $e'\ e''$.

($\Rightarrow$) Suppose $R_{\tau_1 \to \tau_2}(e)$. We show each part of $R_{\tau_1 \to \tau_2}(e')$:

- $\vdash e' : \tau_1 \to \tau_2$, by preservation.
- $e'\Downarrow$: since $e\Downarrow$ and $e \to e'$, determinism gives $e'\Downarrow$.
- Let $R_{\tau_1}(e'')$. Since $R_{\tau_1 \to \tau_2}(e)$, we have $R_{\tau_2}(e\ e'')$. By $(*)$ and the induction hypothesis at τ_2, from left to right, $R_{\tau_2}(e'\ e'')$.

($\Leftarrow$) Suppose $R_{\tau_1 \to \tau_2}(e')$. We show each part of $R_{\tau_1 \to \tau_2}(e)$:

- $\vdash e : \tau_1 \to \tau_2$, by assumption.
- $e\Downarrow$: since $e \to e'$ and $e'\Downarrow$, we have $e\Downarrow$.
- Let $R_{\tau_1}(e'')$. Since $R_{\tau_1 \to \tau_2}(e')$, we have $R_{\tau_2}(e'\ e'')$. By $(*)$ and the induction hypothesis at τ_2, from right to left, $R_{\tau_2}(e\ e'')$. □

By induction on the derivation of $e \to^* e'$, the same holds for the multi-step relation. We will use it in both directions:

- *forward:* if $R_\tau(e)$ and $e \to^* v$, then $R_\tau(v)$;
- *backward:* if $\vdash e : \tau$, $e \to^* e'$, and $R_\tau(e')$, then $R_\tau(e)$.

4.1 The Fundamental Lemma

Now let's try to prove the first version of the fundamental lemma.

Proof attempt. By induction on the derivation of $\vdash e : \tau$. In every case, $\vdash e : \tau$ holds by assumption, so we only need to show the rest of $R_\tau(e)$.

Case T-Var: Impossible, since the typing context is empty.

Case T-Unit: Here $e = ()$ and $\tau = \mathbf{unit}$. Since $()$ is a value, it halts. So $R_{\mathbf{unit}}(())$.

Case T-App: Here $e = e_1\ e_2$, with

$$\vdash e_1 : \tau' \to \tau \qquad \text{and} \qquad \vdash e_2 : \tau'.$$

By the induction hypotheses, $R_{\tau' \to \tau}(e_1)$ and $R_{\tau'}(e_2)$. The definition of $R_{\tau' \to \tau}(e_1)$ says that for every e', if $R_{\tau'}(e')$ then $R_\tau(e_1\ e')$. Taking $e' = e_2$ gives $R_\tau(e_1\ e_2)$.

Compare this with our first attempt: we never needed to look at how $e_1\ e_2$ evaluates. The stronger induction hypothesis for e_1 does all the work.

Case T-Abs: Here $e = \lambda x : \tau_1.\, e_0$ and $\tau = \tau_1 \to \tau_2$, with $x : \tau_1 \vdash e_0 : \tau_2$. We must show:

1. $\vdash e : \tau_1 \to \tau_2$, which holds by assumption;
2. $e\Downarrow$, which holds since e is a value; and
3. for every e', if $R_{\tau_1}(e')$ then $R_{\tau_2}(e\ e')$.

For the third part, let $R_{\tau_1}(e')$. By adequacy, $e' \to^* v'$ for some v', and by closure under evaluation (forward), $R_{\tau_1}(v')$. Then

$$(\lambda x : \tau_1.\, e_0)\ e' \ \to^* \ (\lambda x : \tau_1.\, e_0)\ v' \ \to \ e_0\{v'/x\}.$$

Since $\vdash e\ e' : \tau_2$, closure under evaluation (backward) means it is enough to show

$$R_{\tau_2}(e_0\{v'/x\}).$$

And here we are stuck. The premise of T-Abs is $x : \tau_1 \vdash e_0 : \tau_2$, so e_0 is an *open* expression. The induction hypothesis is only about closed expressions, so it says nothing about e_0, let alone about $e_0\{v'/x\}$.

The solution is to strengthen the lemma so that it applies to open expressions, by closing them with a substitution of related values for their free variables. A *substitution*

$$\gamma = \{v_1/x_1, \ldots, v_k/x_k\}$$

maps variables to closed values. We write $\gamma(e)$ for the result of applying it to e, that is, of substituting each v_i for x_i. Since the values are closed, the order of the substitutions does not matter. We write $\gamma[x \mapsto v]$ for the substitution that maps x to v and is otherwise like γ.

Definition (Related substitutions)**.**

$$\gamma \models \Gamma \quad \text{iff} \quad dom(\gamma) = dom(\Gamma) \text{ and } R_{\Gamma(x)}(\gamma(x)) \text{ for each } x \in dom(\Gamma)$$

Lemma (Fundamental lemma)**.** *If* $\Gamma \vdash e : \tau$ *and* $\gamma \models \Gamma$*, then* $R_\tau(\gamma(e))$*.*

With Γ and γ empty, this is the first version of the lemma.

Proof. By induction on the derivation of $\Gamma \vdash e : \tau$. Each $\gamma(x)$ is a closed value with $\vdash \gamma(x) : \Gamma(x)$. So the substitution lemma gives $\vdash \gamma(e) : \tau$ in every case, and we only need to show the rest of $R_\tau(\gamma(e))$.

Case T-Var: Here $e = x$ and $\tau = \Gamma(x)$. Since $\gamma \models \Gamma$, we have $R_{\Gamma(x)}(\gamma(x))$.

Case T-Unit: Here $e = ()$ and $\tau =$ **unit**. Then $\gamma(e) = ()$, which halts.

Case T-App: Here $e = e_1\ e_2$, with

$$\Gamma \vdash e_1 : \tau' \to \tau \qquad \text{and} \qquad \Gamma \vdash e_2 : \tau'.$$

By the induction hypotheses, $R_{\tau' \to \tau}(\gamma(e_1))$ and $R_{\tau'}(\gamma(e_2))$. By the definition of $R_{\tau' \to \tau}$, we get $R_\tau(\gamma(e_1)\ \gamma(e_2))$. And $\gamma(e_1)\ \gamma(e_2) = \gamma(e_1\ e_2)$.

Case T-Abs: Here $e = \lambda x : \tau_1.\, e_0$ and $\tau = \tau_1 \to \tau_2$, with $\Gamma, x : \tau_1 \vdash e_0 : \tau_2$. By renaming, we can assume that $x \notin dom(\Gamma)$, so

$$\gamma(e) = \lambda x : \tau_1.\, \gamma(e_0).$$

This is a value, so it halts. Now let $R_{\tau_1}(e')$.

- By adequacy, $e' \to^* v'$ for some v', and by closure under evaluation, $R_{\tau_1}(v')$.
- So $\gamma[x \mapsto v'] \models \Gamma, x : \tau_1$, and by the induction hypothesis,

$$R_{\tau_2}(\gamma[x \mapsto v'](e_0)), \quad \text{that is,} \quad R_{\tau_2}(\gamma(e_0)\{v'/x\}).$$

- But $\gamma(e)\ e' \to^* \gamma(e)\ v' \to \gamma(e_0)\{v'/x\}$ and $\vdash \gamma(e)\ e' : \tau_2$. So by closure under evaluation, $R_{\tau_2}(\gamma(e)\ e')$.

Therefore $R_{\tau_1 \to \tau_2}(\gamma(e))$. □

Notice how the induction hypothesis in the abstraction case is used for *every* related value v'. This is why the lemma must quantify over all related substitutions γ.

Theorem (Termination)**.** *If* $\vdash e : \tau$*, then* $e\Downarrow$*.*

Proof. By the fundamental lemma, with Γ and γ empty, we have $R_\tau(e)$. By adequacy, $e\Downarrow$. □

To summarize, a proof by logical relations follows a recipe:

1. Define a relation by induction on types, with the property of interest at base types, and "related arguments to related results" at function types.
2. Prove *adequacy*: related terms have the property.
3. Prove the *fundamental lemma*: well-typed terms are related, generalized to open terms closed by related substitutions.

5 Extensions

5.1 Booleans

Let's add true, false, and if e_1 then e_2 else e_3, with the semantics and typing rules from Lecture 12. We need one new clause in the definition of the logical relation, for the new type:

$$R_{\mathbf{bool}}(e) \quad \text{iff} \quad \vdash e : \mathbf{bool} \text{ and } e\Downarrow$$

The proof of closure under evaluation for **bool** is the same as for **unit**. The fundamental lemma has three new cases. The cases for T-True and T-False are immediate, since the constants are values.

Case T-If: Here $e =$ if e_1 then e_2 else e_3, with

$$\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau.$$

By the induction hypothesis, $R_{\mathbf{bool}}(\gamma(e_1))$, so $\gamma(e_1) \to^* v$ for some v. By preservation and canonical forms, v is true or false. If $v =$ true, then

$$\gamma(e) \;\to^*\; \text{if true then } \gamma(e_2) \text{ else } \gamma(e_3) \;\to\; \gamma(e_2).$$

By the induction hypothesis, $R_\tau(\gamma(e_2))$, so by closure under evaluation, $R_\tau(\gamma(e))$. The case $v =$ false is symmetric.

Notice that we never needed to know anything about the branch that is not taken.

5.2 Products

For products, an expression is related if it halts and both of its projections are related:

$$\begin{aligned} R_{\tau_1 \times \tau_2}(e) \quad \text{iff} \quad & \vdash e : \tau_1 \times \tau_2 \text{ and } e\Downarrow \text{ and} \\ & R_{\tau_1}(\#1\ e) \text{ and } R_{\tau_2}(\#2\ e) \end{aligned}$$

In general, each type constructor gets a clause that says how its values can be *used*: functions are applied, pairs are projected, and Booleans are tested. Sums are similar, and are an exercise on the homework.

5.3 What Breaks

Fixed points. If we add the fixed-point operator fix from Lecture 12, the termination theorem becomes false, so the proof must fail somewhere. It fails in the case for $\mathsf{fix}\,(\lambda f : \tau.\, e)$, which steps to

$$e\{\mathsf{fix}\,(\lambda f : \tau.\, e)/f\}.$$

To use the induction hypothesis for e, we would need to know R_τ of the fix expression itself, which is exactly what we are trying to prove.

Recursive types. With the recursive types from Lecture 13, we cannot even define the logical relation. The clause for $\mu\alpha.\, \tau$ would refer to $R_{\tau\{\mu\alpha.\tau/\alpha\}}$, at a type that is not smaller.

5.4 Strong Normalization

We proved that *call-by-value* evaluation of a well-typed program terminates. The strong normalization theorem from Lecture 12 says that *every* sequence of full β-reductions is finite. The same proof structure works, with "e is strongly normalizing" in place of "$e\Downarrow$". Closure under reduction is more delicate, though, since reduction is no longer deterministic.

For System F, defining the relation is harder. The relation for $\forall X.\, \tau$ has to say what happens for every possible instantiation of X. But X can be instantiated with any type, including $\forall X.\, \tau$ itself, so we cannot define the relation by induction on types. Girard (1972) solved this problem by quantifying over all sets of terms that satisfy certain closure properties, which he called *reducibility candidates*. In the next lecture, we will see a similar idea in a binary logical relation for System F.